Lab 1: The Cryptographic "Tux" Challenge

Visualizing AES Modes, Malleability, and Authenticated Encryption

- **Course:** CSC 4575: Cryptography and Network Security
- **Knowledge Units:** BCY-4 (Modes of Operation), CSP-3 (Security Failures)
- **Tools:** Python 3, `cryptography` library, `matplotlib`
- **Weight:** 100 Points

---

1. Objective

The goal of this lab is to move beyond the mathematical theory of AES and witness the practical security implications of **Block Cipher Modes**. You will observe how the same secure algorithm (AES) can produce either a completely revealed pattern or total randomness depending on the mode chosen. Finally, you will prove why encryption alone does not provide integrity.

---

2. Background Theory

In symmetric encryption, we use three pillars to ensure security:

1. **The Cipher (AES):** Provides confusion via S-boxes and finite field arithmetic in .
2. **The Mode:** Provides diffusion and allows the 128-bit block cipher to handle large files.
3. **Randomness (IV/Nonce):** Ensures that the same plaintext encrypted twice results in different ciphertexts (**Semantic Security**).

The Modes we will test:

- **ECB (Electronic Codebook):** . No randomness. Identical blocks = identical ciphertext.
- **CBC (Cipher Block Chaining):** . Requires a random Initialization Vector (IV).
- **CTR (Counter):** . No padding required.
- **GCM (Galois/Counter Mode):** AEAD (Authenticated Encryption). Provides confidentiality and a "Tag" for integrity.

---

3. Lab Environment Setup

Ensure your Lab 0 virtual environment is active. We will need `matplotlib` for visualization.

```
cd ~/csc4575
source venv/bin/activate
pip install matplotlib
# Download the target image
wget https://upload.wikimedia.org/wikipedia/commons/5/56/Tux.bmp -O
tux.bmp
```

4. Technical Tasks & Scaffolding

Task A: The Comparison Script (`tux_crypto.py`)

Create a script that encrypts `tux.bmp` using four different modes. Note that for BMP files, the first **54 bytes** are the header (metadata like size and color depth). If you encrypt the header, the image viewer will crash. **We must skip the header.**

```
import os
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms,
modes
from cryptography.hazmat.primitives import padding

def pkcs7_pad(data):
    # TODO: Implement manual PKCS#7 padding
    # 1. Calculate how many bytes are needed to reach a multiple of 16
    # 2. Create a bytes object where each byte is that length
    # 3. Return data + padding
    pass

def encrypt_tux(mode_name):
    # Key management: always use CSPRNG
    key = os.urandom(32) # AES-256
    iv = os.urandom(16)
    nonce = os.urandom(12) # Standard for GCM

    with open("tux.bmp", "rb") as f:
        header = f.read(54)
        pixel_data = f.read()

    # Padding logic
    if mode_name in ["ECB", "CBC"]:
        data_to_encrypt = pkcs7_pad(pixel_data)
    else:
        data_to_encrypt = pixel_data # CTR/GCM are stream-based

    # Mode selection logic
    if mode_name == "ECB":
        cipher_mode = modes.ECB()
    elif mode_name == "CBC":
        cipher_mode = modes.CBC(iv)
    elif mode_name == "CTR":
        cipher_mode = modes.CTR(iv) # CTR uses 16-byte nonce/counter
    elif mode_name == "GCM":
        cipher_mode = modes.GCM(nonce)

    # TODO: Initialize the Cipher, create the encryptor,
    # and process the 'data_to_encrypt'.
    # Note: For GCM, you will also get a 'tag'.

    # Save the file
```

```
    with open(f"tux_{mode_name}.bmp", "wb") as f:
        f.write(header + ciphertext[:len(pixel_data)])

# Run for all modes
for m in ["ECB", "CBC", "CTR", "GCM"]:
    encrypt_tux(m)
```

Task B: Visualizing the Results

Since you are working in a VM, use this snippet to generate a comparison plot to include in your report.

```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

def show_comparison():
    files = ['tux.bmp', 'tux_ECB.bmp', 'tux_CBC.bmp', 'tux_CTR.bmp',
'tux_GCM.bmp']
    titles = ['Original', 'ECB (Pattern Leak)', 'CBC', 'CTR', 'GCM']

    plt.figure(figsize=(20,10))
    for i, file in enumerate(files):
        plt.subplot(1, 5, i+1)
        img = mpimg.imread(file)
        plt.imshow(img)
        plt.title(titles[i])
        plt.axis('off')
    plt.savefig('comparison_results.png')
```

Task C: The Bit-Flipping Integrity Test

Prove that CTR mode is "malleable."

```
# CTR Bit-Flip Demo
key = os.urandom(32)
nonce = os.urandom(16)
msg = b"Transfer $000100 to Account #99"

# 1. Encrypt with CTR
cipher = Cipher(algorithms.AES(key), modes.CTR(nonce))
encryptor = cipher.encryptor()
ct = encryptor.update(msg) + encryptor.finalize()

# 2. Tamper: Flip a bit in the ciphertext to change $000100 to $900100
# Remember: CT = PT XOR Keystream.
# Flipping a bit in CT flips the exact same bit in PT upon decryption.
tampered_ct = bytearray(ct)
# TODO: Find the index of the '0' you want to change and XOR it.
```

```
# 3. Decrypt the tampered_ct and print the result.
# 4. Repeat with GCM and observe the 'InvalidTag' error.
```

---

5. Lab Report Instructions

Your report must be a professional PDF document.

1. **Code (Text):** Embed your completed Python scripts directly into the report as text.
2. **Visuals:** Include the `comparison_results.png` generated by your script.
3. **Analysis Questions:**

- **The "Penguin" Problem:** Why is the image still visible in ECB? Explain using the concept of **Deterministic Encryption**.
- **Padding:** Why did `tux_CTR.bmp` not require padding?
- **The Malleability Attack:** In Task C, what happened when you flipped the bit in CTR? If this were a bank transaction, what is the real-world impact?
- **GCM vs CTR:** Explain why GCM caught the tampering while CTR did not. Mention the "Authentication Tag" ().

---

6. Scoring Rubric

Criteria	Points	Requirements
Code Completion	40	`pkcs7_pad` and `Cipher` logic are implemented correctly.
Visual Gallery	20	Plot clearly shows the difference between ECB and the other modes.
Integrity Test	20	Successfully demonstrates a bit-flip in CTR and an exception in GCM.
Analysis	20	Correct usage of terms like "Malleability," "Semantic Security," and "AEAD."
Formatting	**-10**	**Penalty** if code is not embedded as text or images are unreadable.